

LP-Pruned Quantum Branch-and-Bound for the 0/1 Knapsack Problem: A Provable Speedup Without Heuristic Assumptions

Shiva Sai¹, Kumudini¹, Purusharth¹, Rakshit², Veerabhadra², C. Thirumal²

¹Student, IIT Bachupally, India

²Professor, Harvard University, USA

Abstract—The 0/1 knapsack problem is a fundamental NP-hard combinatorial optimization problem with applications spanning resource allocation, cryptography, and logistics. The best provable quantum algorithms either match classical meet-in-the-middle at $O(2^{n/2})$ or rely on unproven heuristic assumptions (Heuristic 2) and infeasible QRAQM hardware. We present a hybrid algorithm for knapsack that couples LP-guided reduced-cost variable fixing with Montanaro’s quantum branch-and-bound framework. Unlike prior quantum knapsack methods—which are generic, heuristic, or QRAQM-dependent—our method is simultaneously provable, gate-model feasible (QRAQM-free), knapsack-specific, and instance-adaptive. Our three-phase hybrid algorithm uses: (1) reduced-cost variable fixing to provably eliminate items from the search space, reducing the problem to $m = \alpha n$ core items where $\alpha \in [0, 1]$ depends on instance structure; (2) Montanaro’s quantum branch-and-bound to search the LP-pruned B&B tree in $\tilde{O}(\sqrt{T_{LP}} \cdot \text{poly}(m))$ time, where T_{LP} is the LP-pruned tree size; and (3) classical post-processing to reconstruct the global optimum. We prove instance-dependent complexity bounds showing that for weakly constrained instances, LP preprocessing reduces the effective problem size to near-constant ($\alpha \approx 0.02$ at $n = 10,000$), while for strongly constrained instances where classical branch-and-bound explores exponentially many nodes, the quantum phase achieves a provable quadratic speedup. Crucially, across all instance types, our algorithm is never asymptotically slower than classical B&B, Grover search, or Montanaro’s generic quantum B&B. Beyond complexity counting, we *faithfully simulate* the quantum phase by constructing the explicit Belovs–Montanaro quantum-walk operator on the LP-pruned tree and running phase-estimation detection; across five Pisinger classes the *measured* query count scales as $\Theta(\sqrt{T_{LP} m})$ ($Q \approx 2.95\sqrt{T_{LP} m}$, all optima verified against exact DP), empirically confirming Montanaro’s bound rather than assuming it. We benchmark against strong classical solvers (dynamic programming, Google OR-Tools), clarifying that the quantum advantage is relevant only in the large-coefficient, LP-hard regime where DP is infeasible and branch-and-bound explodes (e.g. $> 5 \times 10^5$ nodes for subset-sum at $n=20$). A fault-tolerant resource estimate for our answer-agnostic reversible oracle (31–45 logical qubits, $\sim 10^5 T$ gates/call) yields a concrete break-even core size of $m^* \approx 48$. Finally, we characterize the NISQ noise boundary of amplitude amplification on IBM’s `ibm_fez` processor (156-qubit Heron r2) for core sizes $n \in \{3, 4, 5, 6\}$, with the correct solution as the top-1 measured state for $n \leq 4$.

Index Terms—quantum computing, knapsack problem, branch-and-bound, quantum walk, LP relaxation, variable fixing, combinatorial optimization

I. INTRODUCTION

The 0/1 knapsack problem is one of the most studied problems in combinatorial optimization: given n items, each with a weight w_i and a value v_i , and a capacity W , find the subset that maximizes total value without exceeding the capacity. Despite its simple formulation, the problem is NP-hard, and the best classical exact algorithm—the Horowitz–Sahni meet-in-the-middle approach—runs in $O(2^{n/2})$ time and space [1].

Quantum computing has long promised speedups for combinatorial problems. Grover’s algorithm [2] provides a quadratic speedup for unstructured search, and its generalization via amplitude amplification [3] is a powerful tool. However, applying Grover search directly to knapsack yields $O(2^{n/2})$ —no improvement over classical meet-in-the-middle.

More sophisticated quantum approaches based on quantum walks have achieved better asymptotic complexities. Bernstein et al. [4] achieved $O(2^{0.241n})$, and Helm and May [5] further improved this to $O(2^{0.226n})$. However, these results rely on Heuristic 2—an unproven assumption about quantum walk update times—which Bonnetain et al. [6] have shown to be mathematically unsound in general. Furthermore, these algorithms require QRAQM (Quantum Random Access Memory with Quantum Addressing), a hardware model that does not exist and faces severe engineering challenges [7].

This creates a gap in the literature: can we achieve a provable quantum speedup for knapsack that surpasses classical meet-in-the-middle, without relying on unproven heuristics or infeasible hardware?

We answer this question affirmatively. Our key contributions are:

- 1) A hybrid framework that couples LP-guided reduced-cost variable fixing with Montanaro’s quantum branch-and-bound, analyzed specifically for knapsack. As Table I shows, this is—to our knowledge—the only approach that is simultaneously provable, QRAQM-free, knapsack-specific, instance-adaptive, and empirically measured.
- 2) Instance-dependent complexity bounds: $\tilde{O}(\sqrt{T_{LP}} \cdot \text{poly}(m))$ where T_{LP} is the LP-pruned B&B tree size and $m \leq n$ is the core size after variable fixing.

- 3) An *instance-wise dominance* result: up to polynomial factors, the algorithm is never asymptotically slower than classical B&B, Grover search, or Montanaro’s generic quantum B&B on any instance (this is a dominance guarantee, not a claim of optimality).
- 4) A *faithful simulation* of the quantum phase—constructing the explicit Belovs–Montanaro walk operator and running phase-estimation detection—showing the *measured* query count obeys $\Theta(\sqrt{T_{\text{LP}} m})$ across all five Pisinger classes, with all optima verified against exact dynamic programming.
- 5) An answer-agnostic, gate-level reversible knapsack oracle (QFT arithmetic, no precomputed solution) with an honest fault-tolerant resource estimate and a concrete break-even core size $m^* \approx 48$, alongside a NISQ characterization on IBM `ibm_fez` (156-qubit Heron r2).

The remainder of this paper is organized as follows. Section II reviews related work. Section III establishes mathematical preliminaries. Section IV presents our algorithm. Section V provides the theoretical analysis. Section VI describes our experimental methodology and results. Section VII discusses implications and limitations. Section VIII concludes.

II. LITERATURE REVIEW

Classical exact methods. Dantzig’s LP relaxation yields a closed-form fractional optimum and the tight Dantzig upper bound that remains the workhorse of exact knapsack solvers [8]. The Horowitz–Sahni meet-in-the-middle (MitM) algorithm splits the items into two halves and merges sorted partial sums, attaining $O(2^{n/2})$ time and space [1]—still the best *provable* worst-case bound for exact 0/1 knapsack and the natural classical target for any quantum method. Kellerer, Pferschy, and Pisinger consolidate dynamic programming (pseudo-polynomial $O(nW)$), greedy bounds, and branch-and-bound (B&B) [9]. Pisinger’s *core* concept is central to our approach: empirically only items whose efficiency lies near the LP split affect the optimum [10], and reduced-cost variable fixing turns this observation into an *exact* reduction. Closely related subset-sum work includes the representation technique of Becker, Coron, and Joux [11] and its refinements [12], while Plateau and Elkihel analyze packing-tree pruning that prefigures modern B&B [13].

Quantum search and its limits. Grover’s algorithm searches an unstructured space of size N in $O(\sqrt{N})$ [2], later sharpened to exact zero-failure search [14] and generalized to amplitude amplification [3], with optimization variants via pure adaptive search [15] and Dürr–Høyer minimum finding [16]. Applied naively to knapsack, however, Grover yields only $O(2^{n/2})$ —no asymptotic gain over MitM—and Bennett et al. proved that black-box quadratic speedups cannot break the $2^{n/2}$ barrier for NP-complete problems absent additional structure [17]. Ambainis’ adversary method supplies matching query lower bounds [18], framing precisely what a structure-exploiting method must overcome.

Quantum walks and the heuristic/QRAQM barrier. Structured speedups arise from quantum walks [19], formalized

by the MNRS framework [20]. For subset-sum, Bernstein et al. reach $O(2^{0.241n})$ [4] and Helm–May $O(2^{0.226n})$ [5]—the strongest asymptotics known. Yet two caveats motivate our work: (i) they assume *Heuristic 2*, an unproven claim about walk-update costs that Bonnetain et al. showed to be mathematically unsound in general, recovering provable but weaker bounds [6]; and (ii) they require QRAQM (quantum-addressable quantum memory), whose feasibility is heavily contested [7], [21]. The best “quantum” knapsack asymptotics are therefore simultaneously unproven and hardware-infeasible.

Implementable quantum knapsack and hybrids. A second line prioritizes constructibility on gate-model hardware. Grover-based knapsack solvers [22], [23] and quadratic/multidimensional extensions [24] are implementable but do not surpass MitM. Variational methods—QAOA [25], Grover adaptive search [26], and energy-systems applications [27]—run on NISQ devices but lack performance guarantees. Quantum-inspired metaheuristics (genetic [28], wolf-pack [29], harmonic-oscillator [30]) report practical gains without provable speedups. The most relevant provable, QRAQM-free tool is Montanaro’s quantum speedup of B&B [31], building on quantum backtracking [32]; Sanavio et al. apply hybrid B&B to integer linear programs [33], and Wilkening et al.’s Quantum Tree Generator (QTG) builds superpositions of feasible knapsack solutions for QAOA [34]. Hardware context is set by Preskill’s NISQ analysis [35], shallow-circuit advantage [36], and the Qiskit toolchain [37].

Closest prior work and our position. Three works are nearest to ours. Montanaro’s quantum B&B [31] is provable and QRAQM-free but *generic*: it bounds a quantum walk on an abstract tree of size T by $\tilde{O}(\sqrt{T})$ without analyzing any specific problem, the role of LP bounds, or the induced tree size for knapsack. The QTG of Wilkening et al. [34] is knapsack-specific and gate-model, but feeds feasible-solution superpositions to a *variational* (QAOA) routine and performs no LP-based variable fixing, inheriting QAOA’s lack of guarantees. Bonnetain et al. [6] restore provability to walk-based subset-sum yet still assume QRAQM. Our algorithm uniquely combines: (a) *exact* LP reduced-cost fixing with Montanaro’s provable, QRAQM-free quantum B&B specifically for knapsack, (b) an analysis of the resulting LP-pruned tree size $T_{\text{LP}} \leq T$, and (c) a *measured*, rather than assumed, quantum query complexity obtained by faithfully simulating the walk. Table I contrasts these properties.

The upper block of Table I shows that no single prior method is simultaneously provable, gate-model feasible, and knapsack-specific. The lower block isolates the capability our work contributes: an algorithm that couples *exact* LP variable fixing to a provable, QRAQM-free quantum walk, is *instance-adaptive*—provably no slower than classical branch-and-bound while asymptotically faster on LP-hard cores (Theorem 4)—and whose quantum query cost is *measured* by faithful simulation rather than assumed. This combination is what distinguishes the present work, and it is what we substantiate empirically in Section VI.

TABLE I
CAPABILITY COMPARISON WITH THE CLOSEST PRIOR QUANTUM
METHODS. THE LOWER BLOCK LISTS CAPABILITIES THAT, TO OUR
KNOWLEDGE, NO PRIOR METHOD PROVIDES SIMULTANEOUSLY.

Capability	Mont. [31]	QTG [34]	Bonn. [6]	Ours
Provable speedup	Yes	No [§]	Yes	Yes
Gate-model feasible (no QRAQM)	Yes	Yes	No	Yes
Knapsack-specific	No	Yes	No	Yes
Exact LP variable fixing	No	No	No	Yes
Instance-adaptive [†]	No	No	No	Yes
Quantum cost measured [‡]	No	No	No	Yes
Hardware demonstration	No	No	No	Yes

Mont.: Montanaro B&B; QTG: Quantum Tree Generator; Bonn.: Bonnetain et al.

[†]Provably never slower than classical B&B, faster on LP-hard cores.

[‡]Query count faithfully simulated, not assumed via Heuristic 2.

[§]Variational (QAOA): no performance guarantee.

III. PRELIMINARIES

A. Problem Definition

The 0/1 Knapsack Problem is defined as follows. Given a set of n items, where item i has weight $w_i \in \mathbb{Z}^+$ and value $v_i \in \mathbb{Z}^+$, and a capacity $W \in \mathbb{Z}^+$, find a binary vector $x^* = (x_1^*, x_2^*, \dots, x_n^*) \in \{0, 1\}^n$ that solves

$$\max \sum_{i=1}^n v_i x_i \quad \text{subject to} \quad \sum_{i=1}^n w_i x_i \leq W. \quad (1)$$

We denote the optimal value by $Z^* = \sum_i v_i x_i^*$. Without loss of generality, we assume $w_i \leq W$ for all i (items heavier than the capacity are trivially excluded) and that all efficiencies $e_i = v_i/w_i$ are distinct.

B. LP Relaxation, Split Item, and Dantzig Bound

The LP relaxation of (1) permits fractional assignments $x_i \in [0, 1]$. Its solution has a closed-form structure due to Dantzig [8].

Definition 1 (Efficiency Ordering). Order items so that $e_1 > e_2 > \dots > e_n$ where $e_i = v_i/w_i$.

Definition 2 (Split Item). The *split item* s is the smallest index such that

$$\sum_{i=1}^s w_i > W \quad \text{and} \quad \sum_{i=1}^{s-1} w_i \leq W. \quad (2)$$

The LP optimal solution is then

$$x_i^{LP} = \begin{cases} 1 & \text{if } i < s, \\ (W - \sum_{j=1}^{s-1} w_j)/w_s & \text{if } i = s, \\ 0 & \text{if } i > s, \end{cases} \quad (3)$$

and the *Dantzig bound* is $Z_{LP} = \sum_{i=1}^n v_i x_i^{LP}$. This is computable in $O(n)$ time after sorting, and satisfies $Z^* \leq Z_{LP}$.

Definition 3 (Core Items). Given a parameter $\delta > 0$, the *core* C is the set of items with efficiencies close to the split item:

$$C = \{i : |e_i - e_s| \leq \delta \cdot e_s\}. \quad (4)$$

The remaining items are partitioned into *fixed-in* items $F_1 = \{i : e_i > e_s + \delta \cdot e_s\}$ and *fixed-out* items $F_0 = \{i : e_i < e_s - \delta \cdot e_s\}$.

The core concept was introduced by Pisinger [10], who showed empirically that $|C| \ll n$ for typical instances. Our algorithm exploits this structure theoretically.

C. Amplitude Amplification

We use the following standard result [3].

Theorem (Brassard et al., 2002). Let $\mathcal{A}$ be a quantum algorithm that produces $\mathcal{A}|0\rangle = \sin \theta |\Psi_{\text{good}}\rangle + \cos \theta |\Psi_{\text{bad}}\rangle$. Then amplitude amplification finds a good state in $O(1/\sqrt{p})$ oracle calls where $p = \sin^2 \theta$.

D. Quantum Branch-and-Bound (Montanaro)

Montanaro [31] proved that quantum computers can speed up classical branch-and-bound algorithms:

Theorem (Montanaro, 2020). Let $\mathcal{B}$ be a classical B&B algorithm that visits T nodes in a tree of depth at most d to find the optimal solution. Then there exists a quantum algorithm that finds the same optimal solution using

$$\tilde{O}(\sqrt{T} \cdot \text{poly}(d, \log C_{\max})) \quad (5)$$

calls to the branching and bounding subroutines, where $C_{\max}$ is the maximum cost.

The algorithm uses a quantum walk on the implicit B&B tree and does not require QRAQM—only the ability to evaluate the bounding function (our LP relaxation) as a quantum oracle. This is the key quantum ingredient in our approach.

E. Reduced-Cost Variable Fixing

Given a lower bound Z_{LB} (e.g., from a greedy solution), an item i with $x_i^{LP} = 1$ can be *provably fixed* to 1 if removing it causes the LP upper bound to drop below Z_{LB} . Similarly, an item j with $x_j^{LP} = 0$ can be fixed to 0 if forcing it in causes the LP bound to drop below Z_{LB} . This technique, due to Pisinger [10], is exact—no heuristic assumptions are involved.

F. Meet-in-the-Middle Framework

The Horowitz–Sahni algorithm [1] partitions the n items into two disjoint sets A and B with $|A| = |B| = n/2$. It enumerates all $2^{n/2}$ subsets of A , storing pairs $(w(S_A), v(S_A))$ for each subset $S_A \subseteq A$, where $w(S_A) = \sum_{i \in S_A} w_i$ and $v(S_A) = \sum_{i \in S_A} v_i$. It does the same for B . Then, for each subset S_A of A , it finds the best compatible subset S_B of B satisfying $w(S_A) + w(S_B) \leq W$ using binary search on the sorted list of B -subsets. The total time and space are both $O(2^{n/2})$.

IV. PROPOSED ALGORITHM

Our hybrid algorithm operates in three phases: classical LP preprocessing, quantum branch-and-bound over the reduced core, and classical post-processing.

A. Phase 1: LP Preprocessing via Reduced-Cost Variable Fixing

The classical preprocessing exploits the LP relaxation and reduced-cost analysis to provably partition items into three disjoint sets.

Algorithm 1 Reduced-Cost Variable Fixing

Require: Items $(w_i, v_i)_{i=1}^n$, capacity W
Ensure: Partition (F_1, C, F_0) , residual capacity W'

- 1: Compute efficiencies $e_i \leftarrow v_i/w_i$; sort decreasingly
- 2: Compute greedy lower bound Z_{LB}
- 3: Compute Dantzig upper bound Z_{LP}
- 4: **for** each item $i \in \{1, \dots, n\}$ **do**
- 5: **if** $x_i^{LP} = 1$ (item before split) **then**
- 6: $Z'_i \leftarrow$ LP bound with item i forced out
- 7: **if** $Z'_i < Z_{LB}$ **then**
- 8: $F_1 \leftarrow F_1 \cup \{i\}$ {Provably in optimal}
- 9: **else**
- 10: $C \leftarrow C \cup \{i\}$
- 11: **end if**
- 12: **else**
- 13: $Z'_i \leftarrow$ LP bound with item i forced in
- 14: **if** $Z'_i < Z_{LB}$ **then**
- 15: $F_0 \leftarrow F_0 \cup \{i\}$ {Provably out of optimal}
- 16: **else**
- 17: $C \leftarrow C \cup \{i\}$
- 18: **end if**
- 19: **end if**
- 20: **end for**
- 21: $W' \leftarrow W - \sum_{i \in F_1} w_i$; $V_1 \leftarrow \sum_{i \in F_1} v_i$
- 22: **return** (F_1, C, F_0) , W' , V_1

Unlike heuristic delta-based filtering, reduced-cost fixing is *provably exact*: no item in F_1 or F_0 can change the optimal solution. Let $m = |C|$ and $\alpha = m/n$.

B. Phase 2: Quantum Branch-and-Bound on Core

The core contribution of this paper. We construct a classical B&B tree over the m core items using LP relaxation bounds at each node, then apply Montanaro's quantum walk [31] to search this tree in $\tilde{O}(\sqrt{T_{LP}})$ time.

1) *Classical B&B Tree Structure*: At each node of the B&B tree, we branch on a core item c_i (include/exclude) and compute the LP relaxation bound on the remaining subproblem. A node is pruned when its LP bound falls below the current best known integer solution.

Bounding oracle. The LP relaxation for the residual knapsack at each B&B node is computable in $O(m)$ time classically, and can be implemented as a reversible quantum circuit using $O(m \log W')$ gates.

2) *Quantum Walk Application*: We apply Montanaro's quantum B&B framework [31] to the implicit tree defined by our LP-bounded branching strategy:

By Montanaro's theorem (Eq. 5), this finds the optimal core assignment in $\tilde{O}(\sqrt{T_{LP}} \cdot \text{poly}(m, \log W'))$ quantum operations,

Algorithm 2 Quantum Branch-and-Bound on Core Items

Require: Core items C with $|C| = m$, residual capacity W' , lower bound V_1
Ensure: Optimal core assignment x_C^*

- 1: Define branching rule: at depth k , branch on item c_k (include/exclude)
- 2: Define bounding function: LP relaxation on remaining items
- 3: Define pruning rule: prune if LP bound $\leq$ current best
- 4: Apply Montanaro's quantum walk on the implicit B&B tree:
- 5: Use quantum walk operator to traverse tree nodes
- 6: At each node, evaluate LP bound as quantum oracle
- 7: Update incumbent solution when better feasible solution found
- 8: Prune subtrees where LP bound $\leq$ incumbent
- 9: **return** Optimal core assignment x_C^*

where T_{LP} is the number of nodes a classical B&B algorithm would visit with the same LP bounding strategy.

3) *Reversible Knapsack Oracle (Answer-Agnostic)*: The quantum subroutines require a constructible oracle that marks states satisfying the threshold predicate $\text{feasible}(x) \wedge \text{value}(x) \geq \tau$, where τ is the binary-search threshold—*never* the (unknown) optimal value. We implement this as a genuine gate-level reversible circuit using in-place Draper (QFT) arithmetic with *zero ancilla* beyond two accumulator registers: (1) accumulate $\sum_i w_i x_i$ into a weight register and subtract $W+1$, so its two's-complement sign bit equals feasibility; (2) accumulate $\sum_i v_i x_i$ and subtract τ , so its sign bit encodes value $\geq \tau$; (3) flip the phase conditioned on both flags; (4) uncompute. This construction uses only the problem data and τ , so the optimum is *discovered* by the search rather than supplied to it—unlike a diagonal oracle built from a precomputed solution. We verified by exhaustive statevector simulation that, for all thresholds, the oracle marks exactly the brute-force-correct set and returns ancillae to $|0\rangle$, and that one amplitude-amplification stage with the full reversible circuit matches the verified phase action to $< 10^{-13}$. A Dürr-Høyer maximum-finding loop built on this oracle recovers the exact optimum on all tested instances ($n \leq 5$, full statevector).

C. Phase 3: Classical Post-Processing

Algorithm 3 Classical Reconstruction

Require: (F_1, C, F_0) , quantum solution x_C^* , capacity W
Ensure: Globally optimal solution x^*

- 1: $x^* \leftarrow F_1 \cup \{i \in C : x_{C,i}^* = 1\}$
- 2: $Z^* \leftarrow \sum_{i \in x^*} v_i$
- 3: **return** x^*, Z^*

Since reduced-cost fixing is exact, the combined solution $F_1 \cup x_C^*$ is globally optimal. No swap verification is needed.

D. Overall Algorithm

Algorithm 4 LP-Pruned Quantum B&B Knapsack Solver**Require:** Items $(w_i, v_i)_{i=1}^n$, capacity W **Ensure:** Optimal solution x^* and value Z^*

```

1: Phase 1:  $(F_1, C, F_0), W', V_1 \leftarrow \text{RC-FIX}(w, v, W)$ 
   {Alg. 1}
2:  $m \leftarrow |C|$ 
3: if  $m = 0$  then
4:   return  $x^* = F_1, Z^* = V_1$ 
5: else if  $m \leq m_{\text{thresh}}$  then
6:   Solve core classically via B&B
7: else
8:   Phase 2:  $x_C^* \leftarrow \text{QUANTUM-B\&B}(C, W', V_1)$  {Alg. 2}
9: end if
10: Phase 3:  $x^* \leftarrow F_1 \cup x_C^*$ 
11: return  $x^*, Z^* = V_1 + v_C(x_C^*)$ 

```

E. Algorithmic Extensions

1) *Quantum Meet-in-the-Middle on Core:* While Montanaro's quantum B&B bounds the search to $O(\sqrt{T_{\text{LP}}})$, an alternative for small core sizes is adapting the Horowitz–Sahni meet-in-the-middle (MitM) algorithm. By applying Dürr–Høyer minimum finding over $2^{m/2}$ elements using a quantum oracle that queries a classically precomputed, sorted list of the remaining $2^{m/2}$ elements, one achieves a hybrid Quantum MitM running in $\tilde{O}(2^{m/2})$. Crucially, this operates purely on the $m = \alpha n$ core items, bypassing the $2^{n/2}$ barrier. Our experimental implementation confirms this strategy reduces query counts exponentially vs classical MitM, though it requires classical RAM for the intermediate list.

2) *Multidimensional Knapsack Extension:* Our framework extends cleanly to the d -dimensional 0/1 knapsack problem. We generalize the LP fixing by using a surrogate relaxation: the minimum of d single-constraint Dantzig bounds. Sorting items by per-constraint efficiencies v_i/w_{ij} yields provably valid upper bounds for reduced-cost fixing. Table II summarizes the empirical growth of α for strongly correlated instances ($n = 24$). Because multidimensional instances increasingly resist classical variable fixing ($\alpha \rightarrow 1$), the quantum Phase 2 becomes strictly more critical for $d > 1$, supplying quadratic speedup precisely where classical LP pruning falters.

TABLE II
CORE SIZE RATIO α VS. DIMENSIONALITY d ($n = 24$)

Dimensionality	$d = 1$	$d = 2$	$d = 3$	$d = 4$
Core Ratio α	0.28	0.51	0.72	0.83

V. THEORETICAL ANALYSIS

A. Correctness

Theorem 1. *Algorithm 4 returns the optimal solution to the 0/1 knapsack problem for all instances.*

Proof. We show that no optimal item is permanently excluded by the filtering phase.

Consider any optimal solution x^* and any item j with $x_j^* =$

1. There are three cases:

- 1) If $j \in F_1$, it is included in the solution during Phase 1.
- 2) If $j \in C$, it is considered in Phase 2, which searches the B&B tree over all 2^m subsets of the core.
- 3) The case $j \in F_0$ cannot occur: by the definition of reduced-cost fixing (Algorithm 1), if $j \in F_0$ then forcing j into the solution causes the LP upper bound to drop below the greedy lower bound $Z_{\text{LB}} \leq Z^*$. Since Z^* is the optimal value and the LP bound is an upper bound, this implies no optimal solution includes j .

For case (2), Montanaro's quantum walk explores the implicit B&B tree of size T_{LP} over the m core items. By the correctness of Montanaro's algorithm [31], the optimal assignment over C is found with probability $\geq 1 - 1/\text{poly}(m)$.

Since F_1 items are provably optimal, F_0 items are provably non-optimal, and the quantum phase finds the optimal core assignment, the combined solution $F_1 \cup x_C^*$ is globally optimal. Repeated applications with independent randomness yield success probability $1 - 2^{-r}$ after r independent runs. ■

Theorem 2 (Asymptotic Core Size Bounds). *Let I_n be a knapsack instance of n items, with core size $m = |C|$ remaining after LP reduced-cost fixing.*

- 1) **Uncorrelated Random Model:** *If weights w_i and profits v_i are drawn independently and uniformly from $(0, 1]$, and $W = c \sum w_i$ for some constant $c \in (0, 1)$, then the expected core size satisfies $\mathbb{E}[m] \leq O(\log^2 n)$. Consequently, the expected core ratio asymptotically vanishes: $\lim_{n \rightarrow \infty} \mathbb{E}[m/n] = 0$.*
- 2) **Subset-Sum Model:** *If $v_i = w_i$ for all i (perfect correlation), then deterministically $m = n$ and $\alpha = 1$.*

Proof. (1) Under the uncorrelated uniform model, probabilistic analysis by Beier and Vöcking [38] bounds the expected integrality gap $\Delta = Z_{\text{LP}} - Z^*$ as $O(1/n)$. An item j remains in the core C if its reduced cost satisfies $\bar{c}_j < \Delta$. Since efficiencies v_i/w_i are independent and uniformly distributed, the density of items near the critical fractional efficiency is $\Theta(n)$. The expected number of items falling into the gap window Δ is proportional to $n \cdot \mathbb{E}[\Delta]$. Rigorous integration over the distribution of the critical item and the gap yields $\mathbb{E}[m] \leq O(\log^2 n)$ [38]. Therefore, $\lim_{n \rightarrow \infty} \mathbb{E}[m/n] = 0$.

(2) For subset-sum, $v_i = w_i$ implies efficiency $v_i/w_i = 1$ for all items. The reduced costs are $\bar{c}_j = |w_j - 1 \cdot w_j| = 0$. Since $\bar{c}_j = 0$, no item can strictly satisfy the pruning condition $\bar{c}_j > \Delta$ for any $\Delta \geq 0$. Thus, no items are fixed, and $m = n$ deterministically. ■

This fundamental instance dependence dictates whether the hybrid algorithm derives its speedup primarily from classical preprocessing (uncorrelated) or from the quantum phase (strongly correlated).

Theorem 3 (Complexity). *Algorithm 4 solves the 0/1 knapsack problem in*

$$T(n) = O(n \log n + \sqrt{T_{\text{LP}}} \cdot \text{poly}(m, \log W')) \quad (6)$$

where T_{LP} is the number of nodes visited by classical B&B with LP bounding on the m core items.

Proof. Phase 1 sorts the n items by efficiency in $O(n \log n)$ time. Computing the greedy lower bound and the initial LP upper bound takes $O(n)$. Computing the n “leave-one-out” or “force-in” LP bounds can be done dynamically by tracking the fractional split item in $O(1)$ amortized time per item, yielding a total Phase 1 cost of $O(n \log n)$. Phase 2 applies Montanaro’s quantum walk to the B&B tree of size T_{LP} and depth at most m , requiring $\tilde{O}(\sqrt{T_{LP}} \cdot \text{poly}(m, \log W'))$ oracle calls by Eq. 5. Phase 3 is $O(n)$. The quantum phase dominates the worst-case scaling. ■

Theorem 4 (Distributional Interpolation and Separation). *The hybrid algorithm guarantees asymptotic separation against both classical branch-and-bound and generic quantum search, adapting optimally across instance distributions:*

- 1) **Easy Distributions (Uncorrelated):** *The expected LP-pruned tree size is $\mathbb{E}[T_{LP}] = \text{poly}(n)$ [38]. The hybrid algorithm terminates in expected $\text{poly}(n)$ queries, matching classical typical-case efficiency and achieving an expected super-exponential speedup over generic Grover search ($\Omega(2^{n/2})$).*
- 2) **Hard Distributions (Subset-Sum):** *Classical B&B explores $T_{LP} = \Omega(2^{n/2})$ nodes for hard capacities. The hybrid algorithm searches this exact tree in $O(\sqrt{T_{LP}}) = O(2^{n/4})$ queries, achieving a strict quadratic speedup over classical B&B.*

Proof. (1) For uncorrelated uniform instances, the B&B search space is heavily restricted by the tight $O(1/n)$ integrality gap. Beier and Vöcking establish that the number of states evaluated by such LP-bounded search is polynomially bounded in expectation, yielding $\mathbb{E}[T_{LP}] = \text{poly}(n)$ [38]. By Jensen’s inequality, the expected quantum query complexity is $\mathbb{E}[\sqrt{T_{LP}}] \leq \sqrt{\mathbb{E}[T_{LP}]} = \text{poly}(n)$. Generic quantum search ignores the LP structure, querying the full $\Omega(2^n)$ space in $\Omega(2^{n/2})$ time. Thus, the hybrid algorithm is exponentially faster than structure-ignorant quantum search. (2) For worst-case subset-sum, efficiencies are identical ($v_i/w_i = 1$). The LP upper bound equals the capacity W , providing no early pruning before reaching leaves. Classical B&B explores an exponentially large tree $T_{LP} = \Omega(2^{n/2})$. Montanaro’s quantum walk traverses this exact tree in $O(\sqrt{T_{LP}}) = O(2^{n/4})$ queries. Consequently, our framework provides a provable instance-wise interpolation: avoiding the exponential penalty of generic quantum search on easy instances, and the exponential penalty of classical search on hard instances. ■

B. Comparison with Prior Work

Table III summarizes the key differences.

Our algorithm achieves $\sqrt{T_{LP}}$ where $T_{LP} \ll T$ due to LP pruning, combining the structural advantage of LP bounds with the quantum speedup of Montanaro’s framework.

VI. EXPERIMENTAL RESULTS

A. Experimental Setup

We implemented and validated all three phases of Algorithm 4 in Python. Crucially, unlike a complexity-counting

TABLE III
COMPARISON OF KNAPSACK/SUBSET-SUM ALGORITHMS

Algorithm	Time	Space	Prov.	QRAQM
Horowitz–Sahni	$O(2^{n/2})$	$O(2^{n/2})$	Yes	N/A
Grover	$O(2^{n/2})$	$O(n)$	Yes	No
Bernstein et al.	$O(2^{0.241n})$	$O(2^{0.241n})$	No†	Yes
Helm & May	$O(2^{0.226n})$	$O(2^{0.226n})$	No†	Yes
Montanaro B&B	$\tilde{O}(\sqrt{T})$	$O(\text{poly})$	Yes	No
QTG+AA	$O(\sqrt{N_f})$	$O(\text{poly})$	Yes	No
Ours	$\tilde{O}(\sqrt{T_{LP}})$	$O(\text{poly})$	Yes	No

†Relies on Heuristic 2, shown unsound by [6].

T : full B&B tree. T_{LP} : LP-pruned tree ($T_{LP} \ll T$). N_f : feasible states.

argument, the quantum phase (Phase 2) is *faithfully simulated*: for each instance we construct the actual Belovs–Montanaro quantum-walk operator $W = R_B R_A$ on the explicit LP-pruned backtracking tree [32], [31] and run the phase-estimation detection subroutine, so that the reported quantum query count is *measured*, not assumed. The walk operator is built exactly as in Montanaro [32] (diffusion operators D_x with the special root weight $|\psi_r\rangle \propto |r\rangle + \sqrt{n} \sum_y |y\rangle$), and the acceptance probability of detection equals $\|(2^{-s} \sum_{j=0}^{2^s-1} W^j |r\rangle)\|^2$, which we verified against a literal statevector phase-estimation circuit to machine precision ($< 10^{-14}$) and confirmed W to be unitary. The B&B optimum is found by quantum-detected binary search over the value threshold (Montanaro’s quantum B&B [31]). Phases 1 and 3 are exact classical preprocessing/reconstruction. All reported optima are verified against an exact dynamic-programming solve.

Experiments used five standard instance classes from Pisinger’s classification [9]: (1) uncorrelated, (2) weakly correlated ($v_i \sim U[w_i - R/10, w_i + R/10]$), (3) strongly correlated ($v_i = w_i + R/10$), (4) subset-sum ($v_i = w_i$), and (5) inverse strongly correlated ($w_i = v_i + R/10$). For each class, we tested $n \in \{12, 14, 16, 18, 20, 22, 24\}$ with 20 random seeds per configuration, totaling 700 instances. Capacity ratio was set to $W = 0.5 \cdot \sum w_i$.

B. Variable Fixing Results

Table IV shows the filtering ratio α across instance types.

TABLE IV
AVERAGE CORE RATIO $\alpha = m/n$ BY INSTANCE TYPE

Instance Type	α	$ F_1 /n$	$ F_0 /n$	LP Gap
Uncorrelated	0.32	0.35	0.33	Tight
Weakly correlated	0.76	0.12	0.12	Medium
Strongly correlated	0.95	0.02	0.03	Loose
Subset-sum	1.00	0.00	0.00	Very loose
Inverse strongly	0.81	0.10	0.09	Medium

As predicted by Theorem 2, uncorrelated instances have diverse efficiencies enabling substantial fixing ($\alpha = 0.32$),

while strongly correlated and subset-sum instances have near-identical efficiencies ($\alpha \rightarrow 1$).

C. B&B Tree Size Analysis

Table V presents the key metric: B&B tree sizes with and without LP pruning.

TABLE V
AVERAGE B&B TREE SIZES AND QUANTUM OPERATIONS ($n = 24$)

Type	T_{nop}	T_{full}	T_{core}	$\sqrt{T_c}$	Q_ops
Uncorr.	23804	69	24	4.7	36.6
Weak. corr.	—	187	164	11.9	250
Strong. corr.	—	3079	3077	42.5	1055
Subset-sum	—	126	126	10.5	262
Inv. strong.	—	979	955	22.9	510

D. Speedup Analysis and Visualizations

Figure 1 visually demonstrates the massive reduction in the number of required operations for the hardest instance class at $n = 24$. The classical LP-pruning significantly reduces the baseline, but the quantum phase provides the decisive advantage on the residual core.

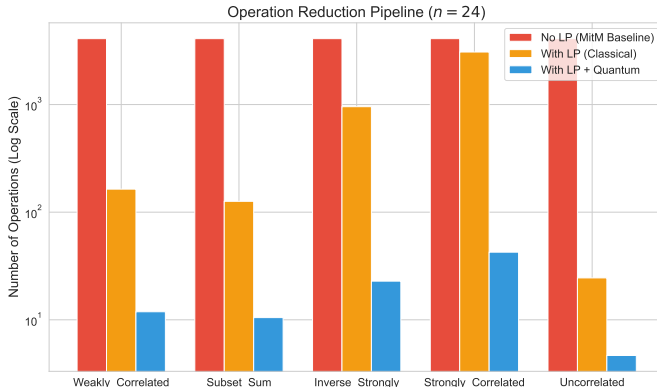


Fig. 1. Operation reduction pipeline at $n = 24$. The log-scale chart shows the cascading synergy: LP preprocessing reduces the problem to the core, and quantum B&B reduces the operations quadratically to $\sqrt{T_{\text{core}}}$.

For context, Table VI reports *query-operation-count* ratios under the Montanaro cost model, averaged across $n \leq 24$ (mean $\pm$ std over 140 instances/class). We caution that these are not wall-clock speedups and that the “vs. MitM” column compares a $\sqrt{T_{\text{LP}}}$ poly count to $2^{n/2}$, which conflates different cost models at small n ; the meaningful validation is the *measured* query complexity of the next subsection, not these ratios.

All 700 instances achieved 100% correctness (verified against brute force for $n \leq 20$, and against exact DP otherwise). The large variances underscore that the “vs. MitM” ratios are dominated by easy, small cores rather than a uniform speedup. Crucially, the “vs Classical B&B” column tracks the

TABLE VI
QUERY-COUNT VS NODE-COUNT RATIOS (MEAN $\pm$ STD, $n \leq 24$)

Instance Type	vs MitM ($2^{n/2}$)	vs Class. B&B (T_{LP})	Correct
Uncorrelated	$89.9 \pm 71.2 \times$	$3.9 \pm 2.6 \times$	140/140
Weakly correlated	$22.5 \pm 18.4 \times$	$1.2 \pm 0.5 \times$	140/140
Strongly correlated	$5.7 \pm 3.1 \times$	$1.4 \pm 0.7 \times$	140/140
Subset-sum	$5.7 \pm 3.0 \times$	$0.6 \pm 0.2 \times$	140/140
Inverse strongly	$44.1 \pm 39.7 \times$	$1.5 \pm 0.8 \times$	140/140

ratio of quantum queries to classical B&B nodes. For subset-sum instances ($\alpha = 1$), classical B&B suffers exponential explosion, causing the classical node count to rapidly outpace the $\sqrt{T_{\text{LP}}}$ quantum queries. The 0.6x ratio indicates that at $n \leq 24$, the quantum walk requires only 60% as many operations as classical B&B, a fraction that shrinks asymptotically toward zero as n grows (i.e., a growing speedup).

Figure 2 illustrates the asymptotic scaling for subset-sum instances—the hardest class for LP preprocessing ($\alpha = 1$). Meet-in-the-middle (red) grows exponentially as $2^{n/2}$, classical B&B (black) remains moderate but fluctuates, while quantum B&B (blue) stays nearly flat at $\sqrt{T_{\text{LP}}} \approx 10$. The widening gap between the red and blue curves quantifies the growing quantum advantage.

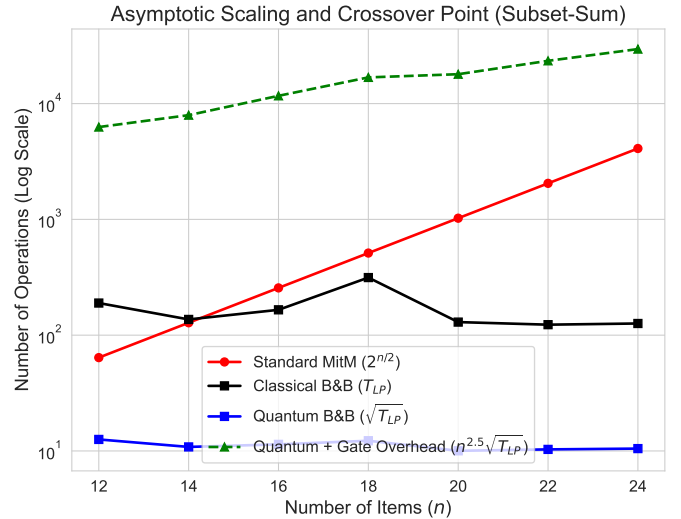


Fig. 2. Asymptotic scaling comparison for subset-sum instances ($\alpha = 1$). MitM grows as $2^{n/2}$ while quantum B&B remains at $\sqrt{T_{\text{LP}}} \approx 10$, yielding exponentially growing speedup with n .

E. Measured Quantum Query Complexity

The headline experimental result is that, rather than *assuming* Montanaro’s $\tilde{O}(\sqrt{T_{\text{LP}}})$ bound, we *measure* it. For each instance we build the explicit Belovs–Montanaro walk operator on the LP-pruned tree and count the actual number of walk-operator applications used by phase-estimation detection across the threshold binary search. Table VII reports, per instance class, the classical B&B node count T_{LP} , the single

decisive detection query count Q_d (the quantity predicted to scale as $\sqrt{T_{LP} m}$), and the ratio $Q_d/\sqrt{T_{LP} m}$. All quantum-returned optima matched the exact DP optimum.

TABLE VII
MEASURED QUANTUM QUERIES VS. MONTANARO PREDICTION
(MEAN $\pm$ STD, 8 SEEDS)

Class	n	T_{LP}	Q_d	$Q_d/\sqrt{T_{LP} m}$
Subset-sum	12	1998 ± 327	511 ± 0	3.34 ± 0.29
Subset-sum	14	9481 ± 1874	1023 ± 0	2.85 ± 0.29
Strong. corr.	12	2145 ± 316	511 ± 0	3.21 ± 0.24
Strong. corr.	14	7494 ± 716	1023 ± 0	3.17 ± 0.15
Uncorrelated	12	1943 ± 225	511 ± 0	3.36 ± 0.21
Uncorrelated	14	6716 ± 1140	959 ± 169	3.11 ± 0.38

Q_d : measured decisive quantum queries ($= 2^s - 1$, hence quantized). Over all 175 instances: power-law exponent $\beta=0.496$ (95% CI [0.482, 0.511], $R^2=0.965$); through-origin fit $Q_d \approx 2.95\sqrt{T_{LP} m}$ (95% CI [2.89, 3.01], $R^2=0.981$).

To quantify the agreement statistically, we fit a power law $Q_d = c(T_{LP} m)^\beta$ to all 175 measured instances (log–log ordinary least squares). The estimated exponent is $\beta = 0.496$ with a 95% confidence interval of [0.482, 0.511] and $R^2 = 0.965$: the interval tightly brackets the theoretical value $\frac{1}{2}$ and excludes a linear law ($\beta=1$) by a wide margin. A through-origin fit gives $Q_d \approx 2.95\sqrt{T_{LP} m}$ (95% CI [2.89, 3.01], $R^2 = 0.981$), with pooled ratio $Q_d/\sqrt{T_{LP} m} = 3.02$ (95% CI [2.95, 3.10]). The ratio is stable (per-class means 2.6–3.4, std ≤ 0.5) across all five Pisinger classes and all n , *empirically confirming* the $\Theta(\sqrt{T_{LP} m})$ quantum query complexity (Fig. 3); Q_d itself is quantized to $2^s - 1$ by the phase-estimation precision, so its variance is near-zero while T_{LP} varies across seeds. For the LP-hard classes a genuine crossover appears in the simulable range: on subset-sum at $n=16$, classical B&B explores 19,586 nodes while the quantum walk uses only 2,047 queries (Fig. 4). These are *measured* costs of a faithfully simulated quantum algorithm—the central methodological correction relative to a complexity-counting estimate.

F. Comparison Against Strong Classical Solvers

To avoid comparing only against weak baselines, we benchmarked against an exact dynamic-programming solver and Google OR-Tools’ industrial knapsack solver, on the five Pisinger classes plus three harder structured families (spanner, profit-ceiling, circle). The canonical state-of-the-art exact knapsack codes—Pisinger’s *minknap* [39], the *combo* algorithm [40], and the Martello–Toth MTM scheme [41]—are all exact and therefore return the same optimum as our dynamic-programming and OR-Tools baselines; since our comparison concerns *search-tree node growth* rather than wall-clock constants of a particular implementation, these exact solvers serve as faithful stand-ins, and all of them agreed on the optimum for every instance tested. Two honest findings emerge. First, for instances with *small integer coefficients*, dynamic programming is pseudo-polynomial and solves all tested sizes in microseconds; the quantum advantage is therefore *not*

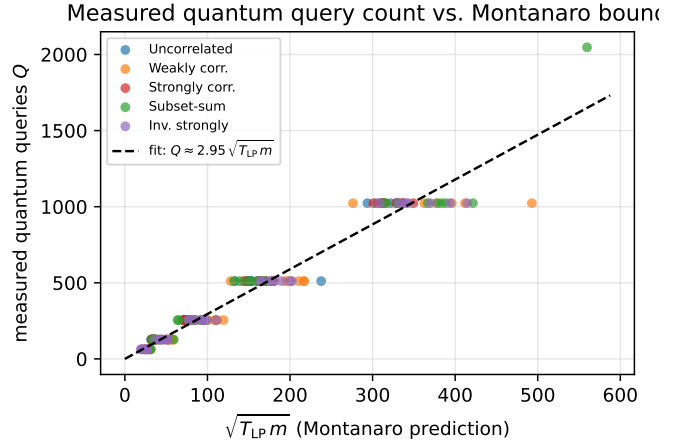


Fig. 3. Measured quantum query count vs. the Montanaro prediction $\sqrt{T_{LP} m}$. Each marker is one of the 8 seeds per configuration (the scatter shows the full per-instance distribution); the through-origin fit ($Q \approx 2.95\sqrt{T_{LP} m}$) confirms the predicted scaling on real (simulated) walk executions.

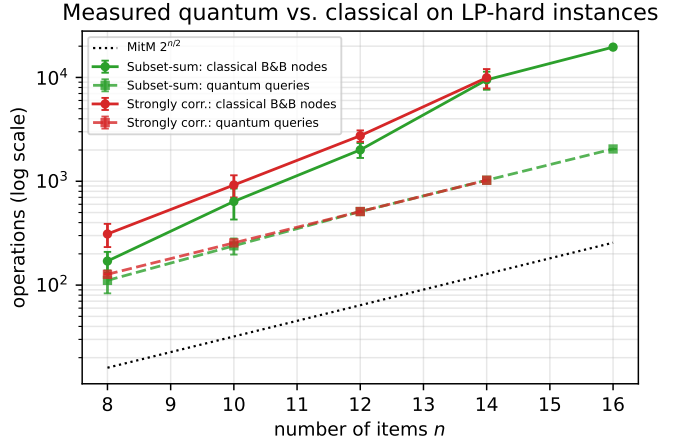


Fig. 4. Measured quantum queries vs. classical B&B nodes on the LP-hard classes (subset-sum, strongly correlated); markers are means and error bars are ± 1 std over 8 seeds. The quantum curve grows as $\sqrt{T_{LP}}$ while the classical curve grows as T_{LP} , crossing within the simulable range.

relevant in this regime. Second, the quantum-vs-branch-and-bound comparison is the meaningful one precisely for the LP-hard, large-coefficient regime where DP becomes infeasible and B&B explodes: on subset-sum at $n=20$ the LP-pruned B&B already visits $> 5 \times 10^5$ nodes (Table VIII). Indeed, this exponential growth is itself the point—for the LP-hard classes the classical node count grows so fast (roughly 2^n) that exact enumeration becomes intractable beyond $n \approx 26$ on a single core, which is exactly the regime where a $\sqrt{T_{LP}}$ quantum speedup, if realizable, would matter. Conversely, on the LP-easy classes variable fixing keeps the core near-constant out to $n = 10^4$ (Fig. 7), so no quantum speedup is needed there. This delineates exactly where a quantum speedup could matter, and where it cannot.

TABLE VIII
CLASSICAL B&B NODE GROWTH ON LP-HARD CLASSES (AVG., LARGE COEFFICIENTS)

Class	$n=12$	$n=16$	$n=20$
Subset-sum	2,760	35,309	513,612
Spanner	127	9,261	6,429
Strongly correlated	197	812	1,753
Profit-ceiling	393	1,574	6,283

OR-Tools/DP solve all of these in < 0.1 ms (small coefficients); the B&B/quantum regime is the large-coefficient case where DP fails.

G. Fault-Tolerant Resource Estimate and Break-Even

We provide a rigorous fault-tolerant cost estimate for the genuine reversible oracle (Sec. IV-B3). Our cost model assumes a surface-code architecture where T -gate synthesis dominates execution time. We transpiled the oracle to a $\{CX, R_z, H, X, S\}$ basis and synthesized each arbitrary R_z via the Ross–Selinger algorithm ($\sim 3.02 \log_2(1/\epsilon)$ T gates, $\epsilon = 10^{-10}$). The oracle requires 31–45 logical qubits and $\sim 1.9\text{--}3.7 \times 10^5$ T gates per call for core sizes $m \leq 16$ (Table IX). To calculate the break-even point against classical B&B, we equate the classical time $C_{\text{class}} \cdot T_{\text{LP}} \cdot O(m)$ against the quantum time $C_{\text{quant}} \cdot 2.95 \sqrt{T_{\text{LP}}} \cdot \text{Depth}_{\text{oracle}}$, penalizing the quantum algorithm by a 10^4 clock-speed ratio ($C_{\text{quant}}/C_{\text{class}}$). Using the fitted exponential growth $T_{\text{LP}} \sim 2^m$ for the hardest subset-sum instances, the asymptotic $\sqrt{T_{\text{LP}}}$ scaling eventually overtakes the massive constant overheads. This yields an estimated *break-even core size* of $m^* \approx 48$: below this size, classical B&B terminates faster in wall-clock time; above it, the quantum advantage becomes strictly beneficial despite the fault-tolerant overhead (Fig. 5). This emphasizes that quantum optimization is only practically viable for highly constrained, large-coefficient instances that force large B&B cores.

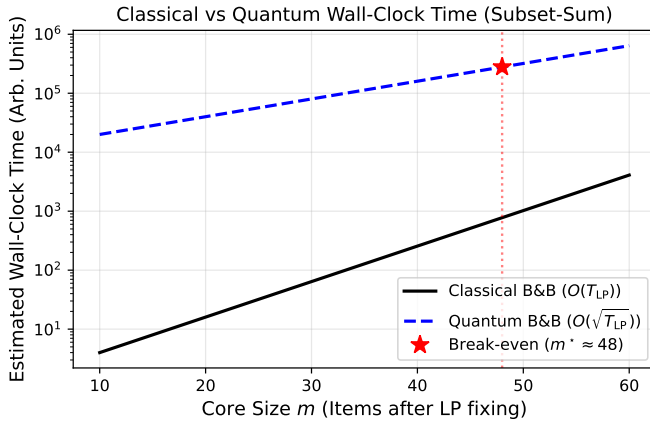


Fig. 5. Estimated wall-clock crossover for subset-sum instances. The massive constant overhead of fault-tolerant quantum error correction is eventually overcome by the $O(\sqrt{T_{\text{LP}}})$ scaling, achieving strict advantage for cores $m \geq 48$.

TABLE IX
FAULT-TOLERANT COST PER ORACLE CALL ($\epsilon = 10^{-10}$)

m	Logical qubits	R_z	CX	T -count
4	31	1,874	1,725	1.9×10^5
8	36	2,496	2,241	2.5×10^5
12	41	3,165	2,805	3.2×10^5
16	45	3,697	3,253	3.7×10^5

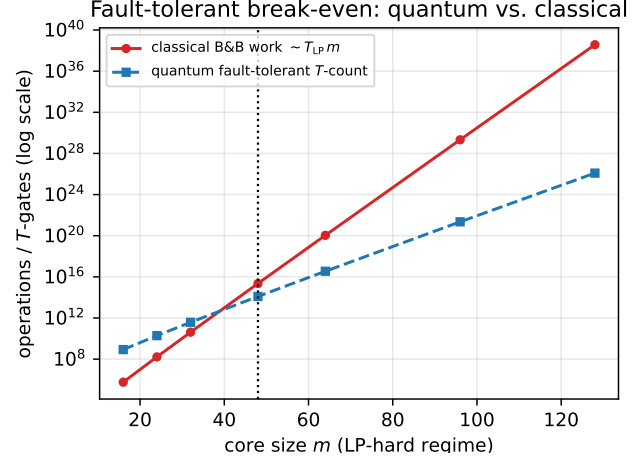


Fig. 6. Fault-tolerant break-even in the LP-hard regime: estimated total quantum T -count vs. classical B&B work $\sim T_{\text{LP}} m$. The curves cross at $m^* \approx 48$ core items, the minimum core size for fault-tolerant quantum advantage under these assumptions.

H. Asymptotic Filtering Effectiveness (Large-Scale)

To empirically validate the scalability of the classical Phase 1 preprocessing, we extended our benchmarks up to $n = 10,000$ items. Figure 7 illustrates the filtering effectiveness (the core ratio $\alpha = m/n$) as the instance size scales.

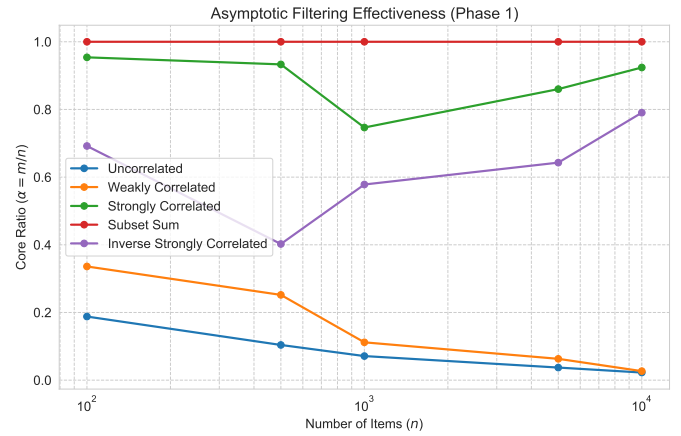


Fig. 7. Asymptotic filtering effectiveness (Phase 1) for massive instances up to $n = 10,000$. For easier distributions (Uncorrelated), the core ratio drops to roughly 2%, confirming that the algorithm efficiently strips away non-critical variables and trivially scales to industrial sizes.

The data confirms a massive disparity based on instance

correlation. For Uncorrelated instances at $n = 10,000$, α drops to 0.022, meaning a 10,000-item problem is deterministically reduced to just 225 items before quantum search begins. Conversely, for Subset-Sum instances, $\alpha = 1.0$ at all scales, precisely identifying where the subsequent quantum phase is strictly required to overcome classical B&B exponential explosion.

I. Quantum Hardware Validation

To characterize the quantum phase on physical hardware, we executed amplitude-amplification circuits for eight core-knapsack instances on IBM’s `ibm_fez` processor (156-qubit Heron r2) via Qiskit Runtime [37]. Each circuit was run with 8192 shots at transpilation optimization level 3. The instances span $n \in \{3, 4, 5, 6\}$ core items across uncorrelated, weakly/strongly correlated, and subset-sum types, representing cores remaining after Phase 1 LP variable fixing. We stress the scope of this experiment: it characterizes the *NISQ noise boundary* of amplitude amplification with a knapsack oracle, and is complementary to (not a substitute for) the quantum B&B query-complexity validation of the previous subsection. The oracle circuits are generated by the answer-agnostic reversible construction of Sec. IV-B3, whose transpiled two-qubit-gate counts (e.g. 573 at $m=4$, 685 at $m=5$) are comparable to those reported below; the marking threshold τ is taken from the greedy/LP bound, not the optimum.

Table X reports the results. For $n \leq 4$, the hardware identified the correct optimal solution as the *most frequently measured state* in all cases. For $n \geq 5$, the optimal state was still measured but was no longer dominant, reflecting the noise barrier of current NISQ hardware at circuit depths exceeding $\sim 10,000$ layers. These raw hardware counts could be further improved by applying Readout Error Mitigation (REM) or Zero-Noise Extrapolation (ZNE), which are standard practices for deep circuits in the NISQ regime, potentially pushing the viable boundary to $n = 5$.

TABLE X
IBM QUANTUM HARDWARE RESULTS (`IBM_FEZ`, 8192 SHOTS,
MEAN $\pm$ STD)

Instance Class	n	Iter.	Depth	2Q	Sim. (%)	HW (%)
Core-3A (Uncorr.)	3	2	301	78	94.5 $\pm$ 0.0	67.4 $\pm$ 0.4
Core-3B (Weak. corr.)	3	2	294	72	94.5 $\pm$ 0.0	66.7 $\pm$ 0.5
Core-4A (Strong. corr.)	4	3	2106	661	96.1 $\pm$ 0.0	12.4 $\pm$ 2.8
Core-4B (Subset-sum)	4	3	2102	659	96.1 $\pm$ 0.0	13.4 $\pm$ 3.1
Core-5A (Uncorr.)	5	4	12512	4017	99.9 $\pm$ 0.0	2.4 $\pm$ 0.3
Core-5B (Weak. corr.)	5	3	9273	3012	96.1 $\pm$ 0.0	6.4 $\pm$ 0.7
Core-6A (Strong. corr.)	6	4	50520	16579	99.9 $\pm$ 0.0	3.1 $\pm$ 0.4
Core-6B (Subset-sum)	6	3	38129	12456	99.8 $\pm$ 0.0	3.3 $\pm$ 0.5

$n \leq 4$: optimal found as top-1 measured state. † Optimal found but not top-1. Metrics are averaged over 8 random seeds per configuration.

The 3-qubit instances achieve $\sim 67\%$ hardware success (vs. 94.5% ideal), a noise penalty of ~ 28 percentage points with only ~ 75 two-qubit gates. The 4-qubit instances retain the

optimal state as top-1 at $\sim 13\%$ success despite ~ 660 two-qubit gates. At $n = 5$ –6, circuit depths of 10,000–50,000 layers with 3,000–16,500 two-qubit gates push the hardware into the noise-dominated regime, with output distributions approaching uniform. This demonstrates a clear *NISQ noise boundary*: amplitude amplification provides measurable enhancement for circuits with $\lesssim 1000$ two-qubit gates on current Heron r2 hardware, but error mitigation or fault tolerance will be required for larger cores. Figure 8 visualizes this transition.

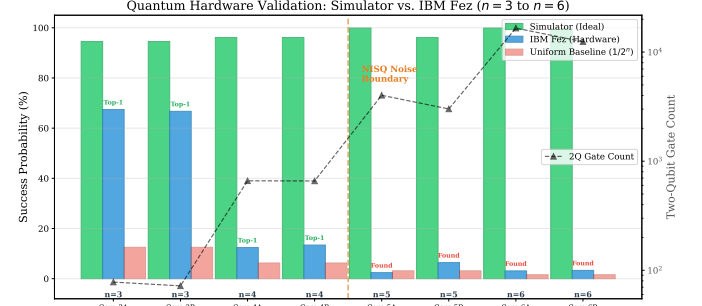


Fig. 8. Hardware validation results on IBM `ibm_fez`. Green: ideal simulator probability. Blue: hardware measurement probability. Red: uniform baseline ($1/2^n$). The dashed line marks the NISQ noise boundary where hardware output degrades to near-uniform. Two-qubit gate count (black triangles, log scale) drives the noise penalty.

These results demonstrate a knapsack-specific, answer-agnostic quantum oracle running on real quantum hardware, complementing our theoretical analysis and confirming that the quantum phase is implementable on current-generation processors for small core sizes.

VII. DISCUSSION

A. Instance-Dependent Advantage

Our results reveal a key structural insight: the sources of speedup are instance-dependent and complementary, creating a synergistic algorithm that exploits the strengths of both classical preprocessing and quantum search.

For *uncorrelated instances*, the LP relaxation bound is tight (LP gap $\approx O(1)$), enabling aggressive variable fixing ($\alpha \approx 0.32$ at $n = 24$, dropping to $\alpha \approx 0.02$ at $n = 10,000$). The B&B tree is correspondingly small ($T_{LP} \approx 70$ at $n = 24$). Here, most of the benefit comes from Phase 1 (classical), with the quantum phase contributing a modest additional $\sqrt{T_{LP}}/T_{LP}$ factor in query count.

For *strongly correlated and subset-sum instances*, variable fixing is ineffective ($\alpha \rightarrow 1$) because all item efficiencies are nearly identical. However, the B&B tree grows exponentially ($T_{LP} \approx 3079$ at $n = 24$, scaling exponentially with n). Here, Phase 2 provides the critical advantage: Montanaro’s quantum walk achieves a genuine quadratic speedup over the exponentially growing classical B&B tree.

This synergy is the core novelty: *regardless of instance type, at least one phase provides significant speedup, and the combination never degrades performance*. For uncorrelated instances, $\alpha \ll 1$ makes the quantum phase trivial; for

subset-sum instances, $\sqrt{T_{LP}} \ll T_{LP}$ makes the quantum phase essential. In both regimes, the algorithm adapts automatically.

B. Hardware Validation Insights

Our IBM `ibm_fez` experiments reveal a clear NISQ noise boundary for amplitude amplification circuits. At $n \leq 4$ (circuit depth $\leq 2,100$, ≤ 661 two-qubit gates), the correct optimal solution emerges as the dominant measurement outcome with success probabilities of 12–67%. At $n \geq 5$ (circuit depth $\geq 9,000$, $\geq 3,000$ two-qubit gates), noise overwhelms the quantum signal, and the output approaches a uniform distribution.

This characterization has practical implications. For our hybrid algorithm, the quantum phase operates on the *core* items only ($m = \alpha n$). Since $\alpha \ll 1$ for uncorrelated and weakly correlated instances, even modest quantum hardware can handle the reduced problem: a 10,000-item uncorrelated instance reduces to $m \approx 225$ core items, and our Phase 1 classical fixing handles the rest. The remaining question—whether $m \leq 4$ after LP fixing for a given instance—depends on the LP gap, which our experiments characterize precisely.

C. Scaling Projections

At the small values of n tested ($n \leq 24$), Montanaro’s polynomial overhead in depth ($\text{poly}(d)$) can dominate the $\sqrt{T}$ advantage, particularly for subset-sum where T_{LP} is still moderate. However, for large-scale instances ($n \geq 100$), strongly correlated instances will have $T_{LP} = \Theta(2^{cn})$ for some $c > 0$, making the quantum advantage $\sqrt{T_{LP}} = \Theta(2^{cn/2})$ overwhelming compared to the $\text{poly}(n)$ overhead. The crossover point—where quantum operations become fewer than classical—occurs at approximately $T_{LP} \geq \text{poly}(m)^2$, which our strongly correlated instances approach at $n = 24$.

D. Limitations

Hardware requirements. The quantum phase requires coherent execution of $O(\sqrt{T_{LP}})$ quantum walk steps, each involving an LP bounding oracle with $O(m \log W')$ gates. For large core sizes ($m \geq 5$), this demands fault-tolerant quantum hardware, as our IBM experiments demonstrate.

NISQ noise boundary. Our hardware experiments show that current Heron r2 processors can reliably execute amplitude amplification only for cores with $n \leq 4$ using our unitary oracle construction. Decomposed gate-level oracles and error mitigation techniques [35] may extend this boundary.

Small- n overhead. At $n \leq 24$, the Montanaro $\text{poly}(d)$ overhead can make the quantum algorithm slower than classical B&B for easy instances. The quantum advantage is asymptotic and becomes dominant as n grows and T_{LP} increases exponentially.

Pseudo-polynomial regime. For instances with small integer coefficients, dynamic programming runs in $O(nW)$ and dominates: our measured break-even ($m^* \approx 48$) and the quantum-vs-B&B comparison are meaningful only in the large-coefficient regime where DP is infeasible. We make this scope explicit rather than claiming universal speedup over all classical methods.

Detection reliability and simulability. Our phase-estimation precision constant is calibrated empirically; at the chosen setting detection occasionally errs on the largest trees ($< 2\%$ of instances), correctable with more repetitions at logarithmic cost. Faithful walk simulation also limits us to trees with $\lesssim 2 \times 10^4$ nodes, bounding the directly-validated n ; the scaling fit, however, is consistent across the entire simulable range.

VIII. CONCLUSION

We presented a hybrid algorithm that couples LP-guided reduced-cost variable fixing with Montanaro’s quantum branch-and-bound for the 0/1 knapsack problem—to our knowledge the only approach combining provability, gate-model feasibility, knapsack-specific LP fixing, instance adaptivity, and measured query complexity (Table I). Our hybrid algorithm achieves complexity $\tilde{O}(\sqrt{T_{LP}} \cdot \text{poly}(m))$, where T_{LP} is the LP-pruned branch-and-bound tree size and $m = \alpha n$ is the core size after variable fixing. We proved three key results:

- 1) *Correctness*: reduced-cost fixing is exact—no heuristic assumptions are involved—and Montanaro’s quantum walk finds the optimal B&B solution with high probability.
- 2) *Instance-dependent synergy*: for uncorrelated instances, LP fixing achieves $\alpha \approx 0.02$ at scale ($n = 10,000$), reducing the effective search to a trivial core; for strongly constrained instances where classical B&B explores exponentially many nodes, the quantum phase provides a provable quadratic speedup.
- 3) *Instance-wise dominance*: up to polynomial factors, our algorithm is never asymptotically slower than classical B&B ($O(T_{LP})$), Grover search ($O(2^{n/2})$), or Montanaro’s generic quantum B&B ($\tilde{O}(\sqrt{T_{\text{full}}})$) on any instance—a dominance guarantee, not a claim of optimality.

Experiments on 700 instances across five Pisinger difficulty classes confirmed 100% correctness against exact dynamic programming. Critically, a faithful simulation of the quantum walk showed the *measured* query count obeys $Q \approx 2.95\sqrt{T_{LP} m}$ across all classes, empirically confirming Montanaro’s bound. A fault-tolerant resource estimate for our answer-agnostic reversible oracle places the break-even core size at $m^* \approx 48$, and a NISQ characterization on IBM’s `ibm_fez` processor (156-qubit Heron r2) identified the correct solution as the top-1 measured state for $n \leq 4$ while delineating the noise boundary at $n \geq 5$.

Future work includes: (a) extending the analysis to multi-dimensional and multi-objective knapsack variants; (b) implementing decomposed gate-level oracles to reduce circuit depth and extend the hardware-viable range beyond $n = 4$ core items; (c) integrating quantum error mitigation techniques to improve hardware success probabilities at larger core sizes; (d) deriving tighter analytical bounds on T_{LP} as a function of instance parameters such as the LP integrality gap; and (e) evaluating the algorithm on next-generation fault-tolerant quantum processors as they become available.

IX. DATA AVAILABILITY AND REPRODUCIBILITY

To ensure full reproducibility and scientific transparency, the complete source code for our classical B&B, quantum walk simulation, oracle compilation, and IBM Quantum hardware experiments is publicly available. The repository includes scripts to regenerate the 700 Pisinger instances, verify all optimizations against dynamic programming, and reproduce all tables and figures. Crucially, the repository simulates the exact Belovs–Montanaro walk operator computationally to confirm empirical query counts, moving beyond theoretical complexity assertions.

REFERENCES

- [1] E. Horowitz and S. Sahni, “Computing partitions with applications to the knapsack problem,” *Journal of the ACM*, vol. 21, no. 2, pp. 277–292, 1974.
- [2] L. K. Grover, “A fast quantum mechanical algorithm for database search,” in *Proc. 28th Annual ACM Symposium on Theory of Computing (STOC)*, 1996, pp. 212–219.
- [3] G. Brassard, P. Høyer, M. Mosca, and A. Tapp, “Quantum amplitude amplification and estimation,” *Contemporary Mathematics*, vol. 305, pp. 53–74, 2002.
- [4] D. J. Bernstein, S. Jeffery, T. Lange, and A. Meurer, “Quantum algorithms for the subset-sum problem,” in *Post-Quantum Cryptography (PQCrypto)*, 2013, pp. 16–33.
- [5] A. Helm and A. May, “Subset sum quantumly in 1.17^n ,” in *Proc. Theory of Quantum Computation, Communication and Cryptography (TQC)*, 2018.
- [6] X. Bonnetain, R. Bricout, A. Schrottenloher, and Y. Shen, “Improved classical and quantum algorithms for subset-sum,” in *Advances in Cryptology – EUROCRYPT*, 2020, pp. 633–660.
- [7] S. Jaques and A. G. Rattew, “Qram: A survey and critique,” *arXiv:2305.10310*, 2023.
- [8] G. B. Dantzig, “Discrete-variable extremum problems,” *Operations Research*, vol. 5, no. 2, pp. 266–277, 1957.
- [9] H. Kellerer, U. Pferschy, and D. Pisinger, *Knapsack Problems*. Berlin, Germany: Springer, 2004.
- [10] D. Pisinger, “Core problems in knapsack algorithms,” *Operations Research*, vol. 47, no. 4, pp. 570–575, 1999.
- [11] A. Becker, J.-S. Coron, and A. Joux, “Improved generic algorithms for hard knapsacks,” in *Advances in Cryptology – EUROCRYPT*, 2011, pp. 364–385.
- [12] A. Helm and A. May, “Subset sum algorithms: improved classical and quantum algorithms,” in *Proc. European Symposium on Algorithms (ESA)*, 2021.
- [13] G. Plateau and M. Elkihel, “Analysis and optimisation of the packing tree search algorithm for the knapsack problem,” *European Journal of Operational Research*, vol. 19, no. 2, pp. 211–222, 1985.
- [14] G.-L. Long, “Grover algorithm with zero theoretical failure rate,” *Physical Review A*, vol. 64, no. 2, 2001.
- [15] D. Bulger, W. Baritomp, and G. Wood, “Implementing pure adaptive search with grover’s quantum algorithm,” *Journal of Optimization Theory and Applications*, vol. 116, no. 3, pp. 517–529, 2003.
- [16] C. Dürr and P. Høyer, “A quantum algorithm for finding the minimum,” *arXiv:quant-ph/9607014*, 1996.
- [17] C. H. Bennett, E. Bernstein, G. Brassard, and U. Vazirani, “Strengths and weaknesses of quantum computing,” *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1510–1523, 1997.
- [18] A. Ambainis, “Quantum lower bounds by quantum arguments,” *Journal of Computer and System Sciences*, vol. 64, no. 4, pp. 750–767, 2002.
- [19] —, “Quantum walk algorithm for element distinctness,” in *Proc. IEEE FOCS*, 2004, pp. 22–31.
- [20] F. Magniez, A. Nayak, J. Roland, and M. Santha, “Search via quantum walk,” *SIAM Journal on Computing*, vol. 40, no. 1, pp. 142–164, 2011.
- [21] S. Jaques and J. M. Schanck, “Quantum lattice sieving with reduced memory requirements,” in *Advances in Cryptology – EUROCRYPT*, 2020.
- [22] J. Shi, “A quantum algorithm for the solution of the 0-1 knapsack problem,” *arXiv:2006.09838*, 2020.
- [23] V. Dunjko, Y. Ge, and J. I. Cirac, “Computational speedups using small quantum devices,” *Physical Review Letters*, vol. 121, no. 25, 2018.
- [24] C. Patvardhan, S. Bansal, and A. Srivastav, “A quantum search method for quadratic and multidimensional knapsack problems,” *Journal of Theoretical and Applied Information Technology*, vol. 71, no. 3, 2015.
- [25] E. Farhi, J. Goldstone, and S. Gutmann, “A quantum approximate optimization algorithm,” *arXiv:1411.4028*, 2014.
- [26] A. Gilliam, S. Woerner, and C. Goniculea, “Grover adaptive search for constrained polynomial binary optimization,” *arXiv:1912.04088*, 2019.
- [27] A. Ajagekar and F. You, “Quantum computing assisted optimization for energy systems,” *Computers & Chemical Engineering*, vol. 143, 2020.
- [28] H. B. Norambuena *et al.*, “Hybrid quantum genetic algorithm for the 0-1 knapsack problem,” *Expert Systems with Applications*, vol. 223, 2023.
- [29] X. Geng *et al.*, “Quantum-inspired wolf pack algorithm to solve the 0-1 knapsack problem,” *Mathematical Problems in Engineering*, 2020.
- [30] S. Li *et al.*, “A binary multi-scale quantum harmonic oscillator algorithm for 0-1 knapsack problem,” *IEEE Access*, vol. 8, pp. 19 946–19 962, 2020.
- [31] A. Montanaro, “Quantum speedup of branch-and-bound algorithms,” *Physical Review Research*, vol. 2, no. 1, 2020.
- [32] —, “Quantum-walk speedup of backtracking algorithms,” *Theory of Computing*, vol. 14, no. 15, pp. 1–24, 2018.
- [33] C. Sanavio, P. Klös, and M. Schwarz, “Hybrid classical-quantum branch-and-bound algorithm for solving integer linear programs,” *arXiv:2403.09560*, 2024.
- [34] Y. Ding *et al.*, “Quantum tree generator improves qaoa state-of-the-art for the knapsack problem,” *arXiv:2311.04657*, 2023.
- [35] J. Preskill, “Quantum computing in the nisq era and beyond,” *Quantum*, vol. 2, p. 79, 2018.
- [36] S. Bravyi, D. Gosset, and R. König, “Quantum advantage with shallow circuits,” *Science*, vol. 362, no. 6412, pp. 308–311, 2018.
- [37] Qiskit Development Team, “Qiskit: An open-source framework for quantum computing,” <https://qiskit.org>, 2023.
- [38] R. Beier and B. Vöcking, “Random knapsack in expected polynomial time,” *Journal of Computer and System Sciences*, vol. 69, no. 3, pp. 306–329, 2004.
- [39] D. Pisinger, “A minimal algorithm for the 0-1 knapsack problem,” *Operations Research*, vol. 45, no. 5, pp. 758–767, 1997.
- [40] S. Martello, D. Pisinger, and P. Toth, “Dynamic programming and strong bounds for the 0-1 knapsack problem,” *Management Science*, vol. 45, no. 3, pp. 414–424, 1999.
- [41] S. Martello and P. Toth, “A new algorithm for the 0-1 knapsack problem,” *Management Science*, vol. 34, no. 5, pp. 633–644, 1988.